Python: Guía Completa de Operadores

Apuntes y notas de referencia sintáctica • Asignación compuesta y comparación

PYTHON 3 CHEATSHEET

> **Concepto clave:** Los símbolos en Python se dividen en operadores de *efecto* (modifican el estado de una variable, como `+=`) y operadores de *evaluación* (producen un valor booleano `True` o `False`, como `!=`).

1. Operadores de Asignación Compuesta

Permiten realizar una operación aritmética o binaria y guardar el resultado directamente en la misma variable.

Operador	Nombre / Operación	Equivalente largo	Ejemplo (`x = 10`)	Valor final
`+=`	Suma y asigna	`x = x + y`	`x += 5`	`15`
`-=`	Resta y asigna	`x = x - y`	`x -= 3`	`7`
`*=`	Multiplica y asigna	`x = x * y`	`x *= 2`	`20`
`/=`	División flotante y asigna	`x = x / y`	`x /= 4`	`2.5`
`//=`	División entera y asigna	`x = x // y`	`x //= 3`	`3`
`%=`	Módulo (resto) y asigna	`x = x % y`	`x %= 3`	`1`
`**=`	Exponente (potencia) y asigna	`x = x ** y`	`x **= 2`	`100`
`&=`	AND binario y asigna	`x = x & y`	`x &= 3`	`2`
`|=`	OR binario y asigna	`x = x | y`	`x |= 3`	`11`
`^=`	XOR binario y asigna	`x = x ^ y`	`x ^= 3`	`9`
`>>=`	Desplazamiento der. y asigna	`x = x >> y`	`x >>= 1`	`5`
`<<=`	Desplazamiento izq. y asigna	`x = x << y`	`x <<= 1`	`20`

2. Operadores de Comparación (Relacionales)

Evalúan relaciones entre dos operandos y devuelven un valor estricto de verdad (`True` o `False`).

Operador	Significado	Forma matemática	Ejemplo	Resultado
`==`	Igual a (igualdad de valor)	$a = b$	`5 == 5`	`True`
`!=`	Distinto de (no igual a)	$a \neq b$	`5 != 3`	`True`
`>`	Estrictamente mayor que	$a > b$	`10 > 4`	`True`
`<`	Estrictamente menor que	$a < b$	`2 < 1`	`False`
`>=`	Mayor o igual que	$a \geq b$	`5 >= 5`	`True`
`<=`	Menor o igual que	$a \leq b$	`3 <= 2`	`False`

3. Operadores Lógicos, Identidad y Pertenencia

Conectan proposiciones booleanas y comprueban membresía o localización en memoria.

Operador	Categoría	Regla lógica / Descripción	Ejemplo	Retorna
`and`	Lógico	`True` únicamente si ambos lados son verdaderos	`(5 > 2) and (3 < 8)`	`True`
`or`	Lógico	`True` si al menos un lado es verdadero	`(5 > 10) or (3 < 8)`	`True`
`not`	Lógico	Invierte el valor de verdad (negación)	`not (10 == 10)`	`False`
`is`	Identidad	Verifica si apuntan a la misma dirección de memoria	`a is None`	`True`
`is not`	Identidad	Verifica si no apuntan al mismo objeto	`x is not None`	`True`
`in`	Pertenencia	`True` si el elemento está en la colección/secuencia	`'py' in 'python'`	`True`
`not in`	Pertenencia	`True` si el elemento no se encuentra dentro	`9 not in [1, 2, 3]`	`True`

4. Distinciones Clave para Evitar Errores Comunes

`=` (Asignación) vs `==` (Comparación)

Un solo signo igual asigna un valor a una variable (`x = 10`). El doble igual evalúa si dos valores son idénticos en magnitud (`x == 10`). Escribir `if x = 10:` generará un `SyntaxError` .

`!=` (Valor) vs `is not` (Identidad)

`!=` compara si el contenido o valor difiere. `is not` compara si son dos instancias distintas en memoria. Dos listas con los mismos elementos `[1, 2] != [1, 2]` es `False` , pero `[1, 2] is not [1, 2]` es `True` .

Comportamiento polimórfico de `+=`

`+=` no solo suma números: concatena cadenas (`s = "a"; s += "b"` da `"ab"`) y muta listas in-place (`l = [1]; l += [2]` añade el elemento sin crear una nueva lista).

Evaluación de Cortocircuito (Short-Circuit)

En `A and B` , si `A` es falso, Python jamás evalúa `B` . En `A or B` , si `A` es verdadero, no se evalúa `B` . Esto previene errores de ejecución como división por cero.

5. Ejemplo Práctico Integrado

```python
# Control de inventario y acceso a pedidos
stock = 15
pedido = 3
usuario_rol = "editor"
roles_autorizados = ["admin", "editor"]

if (usuario_rol in roles_autorizados) and (stock >= pedido) and (pedido != 0):
    stock -= pedido  # Equivale a: stock = stock - pedido (queda en 12)
    print(f"Pedido aprobado. Stock restante: {stock}")
```